

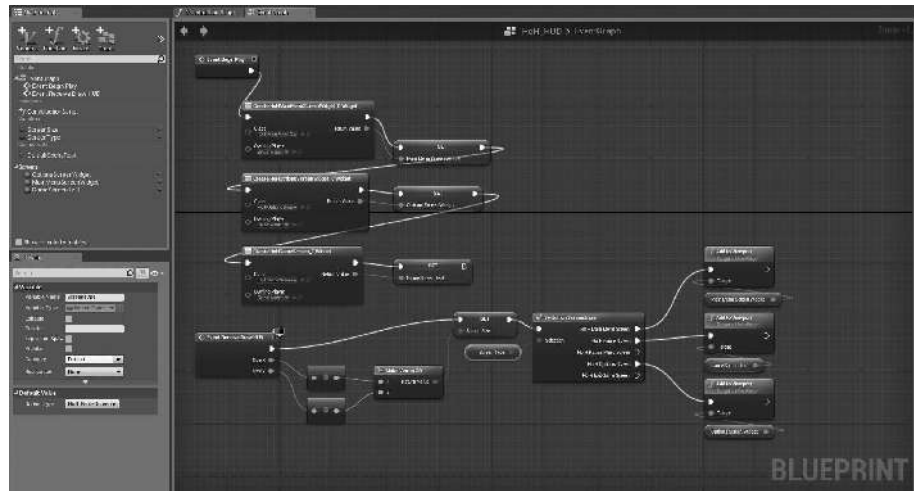


Figure 16.23. Unreal Engine 4's Blueprints.

16.8.11.2 Some Pros and Cons of GUI-Based Programming

The benefits of a graphical user interface over a straightforward, text-file-based scripting language are probably pretty obvious: ease of use, a gradual learning curve with the potential for in-tool help and tool tips to guide the user, and plenty of error-checking. The downsides of a flowchart style GUI include the high cost to develop, debug, and maintain such a system, the additional complexity, which can lead to annoying or sometimes schedule-killing bugs, and the fact that designers are sometimes limited in what they can do with the tool. A text-file-based programming language has some distinct advantages over a GUI-based programming system, including its relative simplicity (meaning that it is much less prone to bugs), the ability to easily search and replace within the source code, and the freedom of each user to choose the text editor with which they are most comfortable.

16.9 Scripting

A *scripting language* can be defined as a programming language whose primary purpose is to permit users to control and customize the behavior of a software application. For example, the Visual Basic language can be used to customize the behavior of Microsoft Excel; both MEL language and Python language can be used to customize the behavior of Maya. In the context of game engines, a

scripting language is a high-level, relatively easy-to-use programming language that provides its users with convenient access to most of the commonly used features of the engine. As such, a scripting language can be used by programmers and non-programmers alike to develop a new game or to customize—or “mod”—an existing game.

16.9.1 Runtime versus Data Definition

We should be careful to make an important distinction here. Game scripting languages generally come in two flavors:

- *Data-definition languages.* The primary purpose of a data-definition language is to permit users to create and populate data structures that are later consumed by the engine. Such languages are often declarative (see below) and are either executed or parsed offline or at runtime when the data is loaded into memory.
- *Runtime scripting language.* Runtime scripting languages are intended to be executed within the context of the engine at runtime. These languages are usually used to extend or customize the hard-coded functionality of the engine’s game object model and/or other engine systems.

In this section, we’ll focus primarily on using a runtime scripting language for the purpose of implementing gameplay features by extending and customizing the game’s object model.

16.9.2 Programming Language Characteristics

In our discussion of scripting languages, it will be helpful for us all to be on the same page with regard to programming language terminology. There are all sorts of programming languages out there, but they can be classified approximately according to a relatively small number of criteria. Let’s take a brief look at these criteria:

- *Interpreted versus compiled languages.* The source code of a *compiled* language is translated by a program called a compiler into machine code, which can be executed directly by the CPU. In contrast, the source code of an *interpreted* language is either parsed directly at runtime or is pre-compiled into platform-independent *byte code*, which is then executed by a *virtual machine* (VM) at runtime. A virtual machine acts like an emulation of an imaginary CPU, and byte code acts like a list of machine

language instructions that are consumed by this virtual CPU. The benefit of a virtual machine is that it can be quite easily ported to almost any hardware platform and embedded within a host application like a game engine. The biggest cost we pay for this flexibility is execution speed—a virtual machine usually executes its byte code instructions much more slowly than the native CPU executes its machine language instructions.

- *Imperative languages.* In an imperative language, a program is described by a sequence of instructions, each of which performs an operation and/or changes the state of data in memory. C and C++ are imperative languages.
- *Declarative languages.* A declarative language describes *what* is to be done but does not specify exactly *how* the result should be obtained. That decision is left up to the people implementing the language. Prolog is an example of a declarative language. Mark-up languages like HTML and TeX can also be classified as declarative languages.
- *Functional languages.* Functional languages, which are technically a subset of declarative languages, aim to avoid state altogether. In a functional language, programs are defined by a collection of functions. Each function produces its results with no side-effects (i.e., it causes no observable changes to the system, other than to produce its output data). A program is constructed by passing input data from one function to the next until the final desired result has been generated. These languages tend to be well-suited to implementing data processing pipelines. They also offer distinct advantages when implementing multithreaded applications, because with no mutable state, a functional language requires no mutex locking. OCaml, Haskell and F# are examples of functional languages.
- *Procedural versus object-oriented languages.* In a procedural language, the primary atom of program construction is the *procedure* (or *function*). These procedures and functions perform operations, calculate results and/or change the state of various data structures in memory. In contrast, an object-oriented language's primary unit of program construction is the *class*, a data structure that is tightly coupled with a set of procedures/functions that "know" how to manage and manipulate the data within that data structure.
- *Reflective languages.* In a reflective language, information about the data types, data member layouts, functions and hierarchical class relationships in the system is available for inspection at runtime. In a non-reflective language, the majority of this meta-information is known only

at compile time; only a very limited amount of it is exposed to the run-time code. C# is an example of a reflective language, while C and C++ are examples of non-reflective languages.

16.9.2.1 Typical Characteristics of Game Scripting Languages

The characteristics that set a game *scripting* language apart from its *native* programming language brethren include:

- *Interpreted.* Most game scripting languages are interpreted by a virtual machine, not compiled. This choice is made in the interest of flexibility, portability and rapid iteration (see below). When code is represented as platform-independent byte code, it can easily be treated like data by the engine. It can be loaded into memory just like any other asset rather than requiring help from the operating system (as is necessary with a DLL on a PC platform or a PRX on the PlayStation 3, for example). Because the code is executed by a virtual machine rather than directly by the CPU, the game engine is afforded a great deal of flexibility regarding how and when script code will be run.
- *Lightweight.* Most game scripting languages have been designed for use in an embedded system. As such, their virtual machines tend to be simple, and their memory footprints tend to be quite small.
- *Support for rapid iteration.* Whenever native code is changed, the program must be recompiled and relinked, and the game must be shut down and rerun in order to see the effects of the changes (unless your development environment supports some form of edit-and-continue). On the other hand, when script code is changed, the effects of the changes can usually be seen very rapidly. Some game engines permit script code to be reloaded on the fly, without shutting down the game at all. Others require the game to be shut down and rerun. But either way, the turnaround time between making a change and seeing its effects in-game is usually much faster than when making changes to the native language source code.
- *Convenience and ease of use.* Scripting languages are often customized to suit the needs of a particular game. Features can be provided that make common tasks simple, intuitive and less error-prone. For example, a game scripting language might provide functions or custom syntax for finding game objects by name, sending and handling events, pausing or manipulating the passage of time, waiting for a specified amount of time to pass, implementing finite state machines, exposing tweakable param-

eters to the world editor for use by the game designers, or even handling network replication for multiplayer games.

16.9.3 Some Common Game Scripting Languages

When implementing a runtime game scripting system, we have one fundamental choice to make: Do we select a third-party commercial or open source language and customize it to suit our needs, or do we design and implement a custom language from scratch?

Creating a custom language from scratch is usually not worth the hassle and the cost of maintenance throughout the project. It can also be difficult or impossible to hire game designers and programmers who are already familiar with a custom, in-house language, so there's usually a training cost as well. However, this is clearly the most flexible and customizable approach, and that flexibility can be worth the investment.

For many studios, it is more convenient to select a reasonably well-known and mature scripting language and extend it with features specific to your game engine. There are a great many third-party scripting languages from which to choose, and many are mature and robust, having been used in a great many projects both within and outside the game industry.

In the following sections, we'll explore a number of custom game scripting languages and a number of game-agnostic languages that are commonly adapted for use in game engines.

16.9.3.1 QuakeC

Id Software's John Carmack implemented a custom scripting language for *Quake*, known as *QuakeC* (QC). This language was essentially a simplified variant of the C programming language with direct hooks into the *Quake* engine. It had no support for pointers or defining arbitrary structs, but it could manipulate *entities* (*Quake*'s name for game objects) in a convenient manner, and it could be used to send and receive/handle game events. QuakeC is an interpreted, imperative, procedural programming language.

The power that QuakeC put into the hands of gamers is one of the factors that gave birth to what is now known as the *mod community*. Scripting languages and other forms of data-driven customization allow gamers to turn many commercial games into all sorts of new gaming experiences—from slight modifications on the original theme to entirely new games.

16.9.3.2 UnrealScript

Probably the best-known example of an entirely custom scripting language is Unreal Engine's *UnrealScript*. This language is based on a C++-like syntactic-

cal style, and it supports most of the concepts that C and C++ programmers have become accustomed to, including classes, local variables, looping, arrays and structs for data organization, strings, hashed string ids (called `FName` in Unreal) and object references (but not free-form pointers). UnrealScript is an interpreted, imperative, object-oriented language.

Epic no longer supports the UnrealScript language. Instead, developers customize game behavior either via the Blueprints graphical “scripting” system, or by writing C++ code.

16.9.3.3 Lua

Lua is a well-known and popular scripting language that is easy to integrate into an application such as a game engine. The Lua website (<http://www.lua.org/about.html>) calls the language the “leading scripting language in games.”

According to the Lua website, Lua’s key benefits are:

- *Robust and mature.* Lua has been used on numerous commercial products, including Adobe’s *Photoshop Lightroom*, and many games, including *World of Warcraft*.
- *Good documentation.* Lua’s reference manual [25] is complete and understandable and is available online and in book formats. A number of books have been written about Lua, including [26] and [50].
- *Excellent runtime performance.* Lua executes its byte code more quickly and efficiently than many other scripting languages.
- *Portable.* Out of the box, Lua runs on all flavors of Windows and UNIX, mobile devices and embedded microprocessors. Lua is written in a portable manner, making it easy to adapt to new hardware platforms.
- *Designed for embedded systems.* Lua’s memory footprint is very small (approximately 350 KiB for the interpreter and all libraries).
- *Simple, powerful and extensible.* The core Lua language is very small and simple, but it is designed to support meta-mechanisms that extend its core functionality in virtually limitless ways. For example, Lua itself is not an object-oriented language, but OOP support can and has been added via a meta-mechanism.
- *Free.* Lua is open source and is distributed under the very liberal MIT license.

Lua is a dynamically typed language, meaning that variables don’t have types—only values do. (Every value carries its type information along with it.) Lua’s primary data structure is the *table*, also known as an associative array. A

table is essentially a list of key-value pairs with an optimized ability to index into the array by key.

Lua provides a convenient interface to the C language—the Lua virtual machine can call and manipulate functions written in C as easily as it can those written in Lua itself.

Lua treats blocks of code, called *chunks*, as first-class objects that can be manipulated by the Lua program itself. Code can be executed in source code format or in precompiled byte code format. This allows the virtual machine to execute a string that contains Lua code, just as if the code were compiled into the original program. Lua also supports some powerful advanced programming constructs, including *coroutines*. This is a simple form of cooperative multitasking, in which each thread must yield the CPU to other threads explicitly (rather than being time-sliced as in a preemptive multithreading system).

Lua does have some pitfalls. For example, its flexible function binding mechanism makes it possible (and quite easy) to redefine an important global function like `sin()` to perform a totally different task (which is usually not something one intends to do). But all in all, Lua has proven itself to be an excellent choice for use as a game scripting language.

16.9.3.4 Python

Python is a procedural, object-oriented, dynamically typed scripting language designed with ease of use, integration with other programming languages, and flexibility in mind. Like Lua, Python is a common choice for use as a game scripting language. According to the official Python website (<http://www.python.org>), some of Python's best features include:

- *Clear and readable syntax.* Python code is easy to read, in part because the syntax enforces a specific indentation style. (It actually parses the whitespace used for indentation in order to determine the scope of each line of code.)
- *Reflective language.* Python includes powerful runtime introspection facilities. Classes in Python are first-class objects, meaning they can be manipulated and queried at runtime, just like any other object.
- *Object-oriented.* One advantage of Python over Lua is that OOP is built into the core language. This makes integrating Python with a game's object model a little easier.
- *Modular.* Python supports hierarchical packages, encouraging clean system design and good encapsulation.

- *Exception-based error handling.* Exceptions make error-handling code in Python simpler, more elegant and more localized than similar code in a non-exception-based language.
- *Extensive standard libraries and third-party modules.* Python libraries exist for virtually every task imaginable. (Really!)
- *Embeddable.* Python can be easily embedded into an application, such as a game engine.
- *Extensive documentation.* There's plenty of documentation and tutorials on Python, both online and in book form. A good place to start is the Python website, <http://www.python.org>.

Python syntax is reminiscent of C in many respects (for example, its use of the = operator for assignment and == for equality testing). However, in Python, *code indentation* serves as the only means of defining *scope* (as opposed to C's opening and closing braces). Python's primary data structures are the *list*—a linearly indexed sequence of atomic values or other nested lists—and the *dictionary*—a table of key-value pairs. Each of these two data structures can hold instances of the other, allowing arbitrarily complex data structures to be constructed easily. In addition, *classes*—unified collections of data elements and functions—are built right into the language.

Python supports *duck typing*, which is a style of dynamic typing in which the functional interface of an object determines its type (rather than being defined by a static inheritance hierarchy). In other words, any class that supports a particular interface (i.e., a collection of functions with specific signatures) can be used interchangeably with any other class that supports that same interface. This is a powerful paradigm: In effect, Python supports polymorphism without requiring the use of inheritance. Duck typing is similar in some respects to C++ template metaprogramming, although it is arguably more flexible because the bindings between caller and callee are formed dynamically, at runtime. Duck typing gets its name from the well-known phrase (attributed to James Whitcomb Riley), "If it walks like a duck and quacks like a duck, I would call it a duck." See http://en.wikipedia.org/wiki/Duck_typing for more information on duck typing.

In summary, Python is easy to use and learn, embeds easily into a game engine, integrates well with a game's object model, and can be an excellent and powerful choice as a game scripting language.

16.9.3.5 Pawn/Small/Small-C

Pawn is a lightweight, dynamically typed, C-like scripting language created by Marc Peter. The language was formerly known as *Small*, which itself was

an evolution of an earlier subset of the C language called *Small-C*, written by Ron Cain and James Hendrix. It is an interpreted language—the source code is compiled into byte code (also known as P-code), which is interpreted by a virtual machine at runtime.

Pawn was designed to have a small memory footprint and to execute its byte code very quickly. Unlike C, Pawn's variables are dynamically typed. Pawn also supports finite state machines, including state-local variables. This unique feature makes it a good fit for many game applications. Good online documentation is available for Pawn (<http://www.compuphase.com/pawn/pawn.htm>). Pawn is open source and can be used free of charge under the Zlib/libpng license (<http://www.opensource.org/licenses/zlib-license.php>).

Pawn's C-like syntax makes it easy to learn for any C/C++ programmer and easy to integrate with a game engine written in C. Its finite state machine support can be very useful for game programming. It has been used successfully on a number of game projects, including *Freaky Flyers* by Midway. Pawn has shown itself to be a viable game scripting language.

16.9.4 Architectures for Scripting

Script code can play all sorts of roles within a game engine. There's a gamut of possible architectures, from tiny snippets of script code that perform simple functions on behalf of an object or engine system to high-level scripts that manage the operation of the game. Here are just a few of the possible architectures:

- *Scripted callbacks.* In this approach, the engine's functionality is largely hard-coded in the native programming language, but certain key bits of functionality are designed to be customizable. This is often implemented via a *hook function* or *callback*—a user-supplied function that is called by the engine for the purpose of allowing customization. Hook functions can be written in the native language, of course, but they can also be written in a scripting language. For example, when updating game objects during the game loop, the engine might call an optional callback function that can be written in script. This gives users the opportunity to customize the way in which the game object updates itself over time.
- *Scripted event handler.* An event handler is really just a special type of hook function whose purpose is to allow a game object to respond to some relevant occurrence within the game world (e.g., responding to an explosion going off) or within the engine itself (e.g., responding to

an out-of-memory condition). Many game engines allow users to write event handler hooks in script as well as in the native language.

- *Extending game object types, or defining new ones, with script.* Some scripting languages allow game object types that have been implemented in the native language to be extended via script. In fact, callbacks and event handlers are examples of this on a small scale, but the idea can be extended even to the point of allowing entirely new types of game objects to be defined in script. This might be done via *inheritance* (i.e., deriving a class written in script from a class written in the native language) or via *composition* (i.e., attaching an instance of a scripted class to a native game object).
- *Scripted components or properties.* In a component- or property-based game object model, it only makes sense to permit new components or property objects to be constructed partially or entirely in script. This approach was used by Gas Powered Games for *Dungeon Siege*. The game object model was property-based, and it was possible to implement properties in either C++ or Gas Powered Games' custom scripting language, *Skrit* (<http://ds.heavengames.com/library/dstk/skrit/skrit>). By the end of the project, they had approximately 148 scripted property types and 21 native C++ property types.
- *Script-driven engine.* Script might be used to drive an entire engine system. For example, the game object model could conceivably be written entirely in script, calling into the native engine code only when it requires the services of lower-level engine components.
- *Script-driven game.* Some game engines actually flip the relationship between the native language and the scripting language on its head. In these engines, the script code runs the whole show, and the native engine code acts merely as a library that is called to access certain high-speed features of the engine. The Panda3D engine (<http://www.panda3d.org>) is an example of this kind of architecture. Panda3D games can be written entirely in the Python language, and the native engine (implemented in C++) acts like a library that is called by script code. (Panda3D games can also be written entirely in C++.)

16.9.5 Features of a Runtime Game Scripting Language

The primary purpose of many game scripting languages is to implement gameplay features, and this is often accomplished by augmenting and customizing a game's object model. In this section, we'll explore some of the most common requirements and features of such a scripting system.

16.9.5.1 Interface with the Native Programming Language

In order for a scripting language to be useful, it must not operate in a vacuum. It's imperative for the game engine to be able to execute script code, and it's usually equally important for script code to be capable of initiating operations within the engine as well.

A runtime scripting language's virtual machine (VM) is generally embedded within the game engine. The engine initializes the virtual machine, runs script code whenever required, and manages those scripts' execution. The unit of execution varies depending on the specifics of the language and the game's implementation.

- In a functional scripting language, the *function* is often the primary unit of execution. In order for the engine to call a script function, it must look up the byte code corresponding to the name of the desired function and spawn a virtual machine to execute it (or instruct an existing VM to do so).
- In an object-oriented scripting language, *classes* are typically the primary unit of execution. In such a system, objects can be spawned and destroyed, and methods (member functions) can be invoked on individual class instances.

It's usually beneficial to allow two-way communication between script and native code. Therefore, most scripting languages allowing native code to be invoked from script as well. The details are language- and implementation-specific, but the basic approach is usually to allow certain script functions to be implemented in the native language rather than in the scripting language. To call an engine function, script code simply makes an ordinary function call. The virtual machine detects that the function has a native implementation, looks up the corresponding native function's address (perhaps by name or via some other kind of unique function identifier), and calls it. For example, some or all of the member functions of a Python class or module can be implemented using C functions. Python maintains a data structure, known as a *method table*, that maps the name of each Python function (represented as a string) to the address of the C function that implements it.

Case Study: Naughty Dog's DC Language

As an example, let's have a brief look at how Naughty Dog's runtime scripting language, a language called DC, was integrated into the engine.

DC is a variant of the Scheme language (which is itself a variant of Lisp language). Chunks of executable code in DC are known as *script lambdas*, which

are the approximate equivalent of functions or code blocks in the Lisp family of languages. A DC programmer writes script lambdas and identifies them by giving them globally unique names. The DC compiler converts these script lambdas into chunks of byte code, which are loaded into memory when the game runs and can be looked up by name using a simple functional interface in C++.

Once the engine has a pointer to a chunk of script lambda byte code, it can execute the code by calling a “virtual machine execution” function in the engine and passing the byte code pointer to it. The function itself is surprisingly simple. It spins in a loop, reading byte code instructions one-by-one, and executing each instruction. When all instructions have been executed, the function returns.

The virtual machine contains a bank of registers, which can hold any kind of data the script may want to deal with. This is implemented using a *variant* data type—a union of all the data types (see Section 16.8.4 for a discussion of variants). Some instructions cause data to be loaded into a register; others cause the data held in a register to be looked up and used. There are instructions for performing all of the mathematical operations available in the language, as well as instructions for performing conditional checks—implementations of DC’s (`if ...`), (`when ...`) and (`cond ...`) instructions and so on.

The virtual machine also supports a *function call stack*. Script lambdas in DC can call other script lambdas (i.e., functions) that have been defined by a script programmer via DC’s (`defun ...`) syntax. Just like any procedural programming language, a stack is needed to keep track of the states of the registers and the return address when one function calls another. In the DC virtual machine, the call stack is literally a stack of register banks—each new function gets its own private bank of registers. This prevents us from having to save off the state of the registers, call the function, and then restore the registers when the called function returns. When the virtual machine encounters a byte code instruction that tells it to call another script lambda, the byte code for that script lambda is looked up by name, a new stack frame is pushed, and execution continues at the first instruction of that script lambda. When the virtual machine encounters a return instruction, the stack frame is popped from the stack, along with the return “address” (which is really just the index of the byte code instruction in the calling script lambda after the one that called the function in the first place).

The following pseudocode should give you a feel for what the core instruction-processing loop of the DC virtual machine looks like:


```

void DcExecuteScript(DCByteCode* pCode)
{
    DCStackFrame* pCurStackFrame
        = DcPushStackFrame(pCode);

    // Keep going until we run out of stack frames (i.e.,
    // the top-level script lambda "function" returns).
    while (pCurStackFrame != nullptr)
    {
        // Get the next instruction. We will never run
        // out, because the return instruction is always
        // last, and it will pop the current stack frame
        // below.
        DCInstruction& instr
            = pCurStackFrame->GetNextInstruction();

        // Perform the operation of the instruction.
        switch (instr.GetOperation())
        {
            case DC_LOAD_REGISTER_IMMEDIATE:
            {
                // Grab the immediate value to be loaded
                // from the instruction.
                Variant& data = instr.GetImmediateValue();

                // Also determine into which register to
                // put it.
                U32 iReg = instr.GetDestRegisterIndex();

                // Grab the register from the stack frame.
                Variant& reg
                    = pCurStackFrame->GetRegister(iReg);

                // Store the immediate data into the
                // register.
                reg = data;
            }
            break;

            // Other load and store register operations...

            case DC_ADD_REGISTERS:
            {
                // Determine the two registers to add. The
                // result will be stored in register A.
                U32 iRegA = instr.GetDestRegisterIndex();
                U32 iRegB = instr.GetSrcRegisterIndex();
            }
        }
    }
}

```

```

        // Grab the 2 register variants from the
        // stack.
        Variant& dataA
            = pCurStackFrame->GetRegister(iRegA);

        Variant& dataB
            = pCurStackFrame->GetRegister(iRegB);

        // Add the registers and store in
        // register A.
        dataA = dataA + dataB;
    }
    break;

// Other math operations...

case DC_CALL_SCRIPT_LAMBDA:
    {
        // Determine in which register the name of
        // the script lambda to call is stored.
        // (Presumably it was loaded by a previous
        // load instr.)
        U32 iReg = instr.GetSrcRegisterIndex();

        // Grab the appropriate register, which
        // contains the name of the lambda to call.
        Variant& lambda
            = pCurStackFrame->GetRegister(iReg);

        // Look up the byte code of the lambda by
        // name.
        DCByteCode* pCalledCode
            = DcLookUpByteCode(lambda.AsStringId());

        // Now "call" the lambda by pushing a new
        // stack frame.
        if (pCalledCode)
        {
            pCurStackFrame
                = DcPushStackFrame(pCalledCode);
        }
    }
    break;

case DC_RETURN:
    {

```

```

        // Just pop the stack frame. If we're in
        // the top lambda on the stack, this
        // function will return nullptr, and the
        // loop will terminate.
        pCurStackFrame = DcPopStackFrame();
    }
    break;

    // Other instructions...

    // ...

    } // end switch
} // end while
}

```

In the above example, we assume that the global functions `DcPushStackFrame()` and `DcPopStackFrame()` manage the stack of register banks for us in some suitable way and that the global function `DcLookUpByteCode()` is capable of looking up any script lambda by name. We won't show implementations of those functions here, because the purpose of this example is simply to show how the inner loop of a script virtual machine might work, not to provide a complete functional implementation.

DC script lambdas can also call native functions—i.e., global functions written in C++ that serve as hooks into the engine itself. When the virtual machine comes across an instruction that calls a native function, the address of the C++ function is looked up by name using a global table that has been hard-coded by the engine programmers. If a suitable C++ function is found, the arguments to the function are taken from registers in the current stack frame, and the function is called. This implies that the C++ function's arguments are always of type `Variant`. If the C++ function returns a value, it too must be a `Variant`, and its value will be stored into a register in the current stack frame for possible use by subsequent instructions.

The global function table might look something like this:

```

typedef Variant DcNativeFunction(U32 argCount,
                                   Variant* aArgs);

struct DcNativeFunctionEntry
{
    StringId          m_name;
    DcNativeFunction* m_pFunc;
};

```

```
DcNativeFunctionEntry g_aNativeFunctionLookupTable[] =
{
    { SID("get-object-pos"), DcGetObjectPos },
    { SID("animate-object"), DcAnimateObject },
    // etc.
};
```

A native DC function implementation might look something like the following. Notice how the Variant arguments are passed to the function as an array. The function must verify that the number of arguments passed to it equals the number of arguments it expects. It must also verify that the types of the argument(s) are as expected and be prepared to handle errors that the DC script programmer may have made when calling the function. At Naughty Dog, we wrote an argument iterator which allows us to extract and verify the arguments one by one in a convenient manner.

```
Variant DcGetObjectPos(U32 argCount, Variant* aArgs)
{
    // Argument iterator expecting at most 2 args.
    DcArgIterator args(argCount, aArgs, 2);

    // Set up a default return value.
    Variant result;
    result.SetAsVector(Vector(0.0f, 0.0f, 0.0f));

    // Use iterator to extract the args. It flags missing
    // or invalid arguments as errors automatically.
    StringId objectName = args.NextStringId();
    Point* pDefaultPos = args.NextPoint(kDcOptional);

    GameObject* pObject
        = GameObject::LookUpByName(objectName);
    if (pObject)
    {
        result.SetAsVector(pObject->GetPosition());
    }
    else
    {
        if (pDefaultPos)
        {
            result.SetAsVector(*pDefaultPos);
        }
        else
        {
            DcErrorMessage("get-object-pos: "
                           "Object '%s' not found.\n",
                           objectName.ToDebugString());
        }
    }
}
```

```
    }  
  
    return result;  
}
```

Note that the function `StringId::ToDebugString()` performs a reverse look-up to convert a string id back to its original string. This requires the game engine to maintain some kind of database mapping each string id to its original string. During development such a database can make life much easier, but because it consumes a lot of memory, the database should be omitted from the final shipped product. (The function name `ToDebugString()` reminds us that the reverse conversion from string id back to string should only be performed for debugging purposes—the game itself must *never* rely on this functionality!)

16.9.5.2 Game Object References

Script functions often need to interact with game objects, which themselves may be implemented partially or entirely in the engine’s native language. The native language’s mechanisms for referencing objects (e.g., pointers or references in C++) won’t necessarily be valid in the scripting language. (It may not support pointers at all, for example.) Therefore, we need to come up with some reliable way for script code to reference game objects.

There are a number of ways to accomplish this. One approach is to refer to objects in script via opaque numeric *handles*. The script code can obtain object handles in various ways. It might be passed a handle by the engine, or it might perform some kind of query, such as asking for the handles of all game objects within a radius of the player or looking up the handle that corresponds to a particular object name. The script can then perform operations on the game object by calling native functions and passing the object’s handle as an argument. On the native language side, the handle is converted back into a pointer to the native object, and then the object can be manipulated as appropriate.

Numeric handles have the benefit of simplicity and should be easy to support in any scripting language that supports integer data. However, they can be unintuitive and difficult to work with. Another alternative is to use the names of the objects, represented as strings, as our handles. This has some interesting benefits over the numeric handle technique. For one thing, strings are human-readable and intuitive to work with. There is a direct correspondence to the names of the objects in the game’s world editor. In addition, we can choose to reserve certain special object names and give them “magic”

meanings. For example, in Naughty Dog’s scripting language, the reserved name “self” always refers to the object to which the currently running script is attached. This allows game designers to write a script, attach it to an object in the game, and then use the script to play an animation on the object by simply writing (`animate 'self name-of-animation`).

Using strings as object handles has its pitfalls, of course. Strings typically occupy more memory than integer ids. And because strings vary in length, dynamic memory allocation is required in order to copy them. String comparisons are slow. Script programmers are apt to make mistakes when typing the names of game objects, which can lead to bugs. In addition, script code can be broken if someone changes the name of an object in the game world editor but forgets to update the name of the object in script.

Hashed string ids overcome most of these problems by converting any strings (regardless of length) into an integer. In theory, hashed string ids enjoy the best of both worlds—they can be read by users just like strings, but they have the runtime performance characteristics of an integer. However, for this to work, your scripting language needs to support hashed string ids in some way. Ideally, we’d like the script compiler to convert our strings into hashed ids for us. That way, the runtime code doesn’t have to deal with the strings at all, only the hashed ids (except possibly for debugging purposes—it’s nice to be able to see the string corresponding to a hashed id in the debugger). However, this isn’t always possible in all scripting languages. Another approach is to allow the user to use strings in script and convert them into hashed ids at runtime, whenever a native function is called.

Naughty Dog’s DC scripting language leverages the concept of “symbols,” which are native to the Scheme programming language, to encode its string ids. Writing `'foo`—or more verbosely, `(quote foo)`—in DC/Scheme corresponds to the string id `SID("foo")` in C++.

16.9.5.3 Receiving and Handling Events in Script

Events are a ubiquitous communication mechanism in most game engines. By permitting event handler functions to be written in script, we open up a powerful avenue for customizing the hard-coded behavior of our game.

Events are usually sent to individual objects and handled within the context of that object. Hence, scripted event handlers need to be associated with an object in some way. Some engines use the game object type system for this purpose—scripted event handlers can be registered on a per-object-type basis. This allows different types of game objects to respond in different ways to the same event but ensures that all instances of each type respond in a consistent and uniform way. The event handler functions themselves can be simple script functions, or they can be members of a class if the scripting language is

object-oriented. In either case, the event handler is typically passed a handle to the particular object to which the event was sent, much as C++ member functions are passed the `this` pointer.

In other engines, scripted event handlers are associated with individual object instances rather than with object types. In this approach, different instances of the same type might respond differently to the same event.

There are all sorts of other possibilities, of course. For example, in Naughty Dog's engine (used to create the *Uncharted* and *The Last of Us* series), scripts are objects in their own right. They can be associated with individual game objects, they can be attached to regions (convex volumes that are used to trigger game events), or they can exist as stand-alone objects in the game world. Each script can have multiple states (that is, scripts are finite state machines in the Naughty Dog engine). In turn, each state can have one or more event handler code blocks. When a game object receives an event, it has the option of handling the event in native C++. It also checks for an attached script object, and if one is found, the event is sent to that script's current state. If the state has an event handler for the event, it is called. Otherwise, the script simply ignores the event.

16.9.5.4 Sending Events

Allowing scripts to handle game events that are generated by the engine is certainly a powerful feature. Even more powerful is the ability to generate and send events from script code either back to the engine or to other scripts.

Ideally, we'd like to be able not only to send predefined types of events from script but to define entirely new event types in script. Implementing this is trivial if event types are strings or string ids. To define a new event type, the script programmer simply comes up with a new event type name and types it into his or her script code. This can be a highly flexible way for scripts to communicate with one another. Script A can define a new event type and send it to Script B. If Script B defines an event handler for this type of event, we've implemented a simple way for Script A to "talk" to Script B. In some game engines, event- or message-passing is the only supported means of inter-object communication in script. This can be an elegant yet powerful and flexible solution.

16.9.5.5 Object-Oriented Scripting Languages

Some scripting languages are inherently object-oriented. Others do not support objects directly but provide mechanisms that can be used to implement classes and objects. In many engines, gameplay is implemented via an object-

oriented game object model of some kind. So it makes sense to permit some form of object-oriented programming in script as well.

Defining Classes in Scripts

A class is really just a bunch of data with some associated functions. So any scripting language that permits new data structures to be defined, and provides some way to store and manipulate functions, can be used to implement classes. For example, in Lua, a class can be built out of a table that stores data members and member functions.

Inheritance in Script

Object-oriented languages do not necessarily support inheritance. However, if this feature is available, it can be extremely useful, just as it is in native programming languages like C++.

In the context of game scripting languages, there are two kinds of inheritance: deriving scripted classes from other scripted classes and deriving scripted classes from native classes. If your scripting language is object-oriented, chances are the former is supported out of the box. However, the latter is tough to implement even if the scripting language supports inheritance. The problem is bridging the gap between two languages and two low-level object models. We won't get into the details of how this might be implemented here, as the implementation is bound to be specific to the pair of languages being integrated. UnrealScript is the only scripting language I've encountered that allows scripted classes to derive from native classes in a seamless way.

Composition/Aggregation in Script

We don't need to rely on inheritance to extend a hierarchy of classes—we can also use composition or aggregation to similar effect. In script, then, all we really need is a way to define classes and associate instances of those classes with objects that have been defined in the native programming language. For example, a game object could hold a pointer or reference to an optional component written entirely in script. We can delegate certain key functionality to the script component, if it exists. The script component might have an `Update()` function that is called whenever the game object is updated, and the scripted component might also be permitted to register some of its member functions/methods as event handlers. When an event is sent to the game object, it calls the appropriate event handler on the scripted component, thus giving the script programmer an opportunity to modify or extend the behavior

of the natively implemented game object.

16.9.5.6 Scripted Finite State Machines

Many problems in game programming can be solved naturally using finite state machines (FSMs). For this reason, some engines build the concept of FSMs right into the core game object model. In such engines, every game object can have one or more states, and it is the states—not the game object itself—that contain the update function, event handler functions and so on. Simple game objects can be created by defining a single state, but more complex game objects have the freedom to define multiple states, each with a different update and event-handling behavior.

If your engine supports a state-driven game object model, it makes a lot of sense to provide finite state machine support in the scripting language as well. And of course, even if the core game object model doesn't support finite state machines natively, one can still provide state-driven behavior by using a state machine on the script side. An FSM can be implemented in any programming language by using class instances to represent states, but some scripting languages provide tools especially for this purpose. An object-oriented scripting language might provide custom syntax that allows a class to contain multiple states, or it might provide tools that help the script programmer easily aggregate state objects together within a central hub object and then delegate the update and event-handling functions to it in a straightforward way. But even if your scripting language provides no such features, you can always adopt a methodology for implementing FSMs and follow those conventions in every script you write.

16.9.5.7 Multithreaded Scripts

It's often useful to be able to execute multiple scripts in parallel. This is especially true on today's highly parallelized hardware architectures. If multiple scripts can run at the same time, we are in effect providing *parallel threads of execution* in script code, much like the threads provided by most multitasking operating systems. Of course, the scripts may not actually run in parallel—if they are all running on a single CPU, the CPU must take turns executing each one. However, from the point of view of the script programmer, the paradigm is one of parallel programming.

Most scripting systems that provide parallelism do so via *cooperative multitasking*. This means that a script will execute until it explicitly yields to another script. This is in contrast with a *preemptive multitasking* approach, in which the execution of any script could be interrupted at any time to permit another

script to execute.

One simple approach to cooperative multitasking in script is to permit scripts to explicitly go to sleep, waiting for something relevant to happen. A script might wait for a specified number of seconds to elapse, or it might wait until a particular event is received. It might wait until another thread of execution has reached a predefined synchronization point. Whatever the reason, whenever a script goes to sleep, it puts itself on a list of sleeping script threads and tells the virtual machine that it can start executing another eligible script. The system keeps track of the conditions that will wake up each sleeping script—when one of these conditions becomes true, the script or scripts waiting on the condition are woken up and allowed to continue executing.

To see how this works in practice, let's look at an example of a multi-threaded script. This script manages the animations of two characters and a door. The two characters are instructed to walk up to the door—each one might take a different, and unpredictable, amount of time to reach it. We'll put the script's threads to sleep while they wait for the characters to reach the door. Once they both arrive at the door, one of the two characters opens the door, which it does by playing an "open door" animation. Note that we don't want to hard-code the duration of the animation into the script itself. That way, if the animators change the animation, we won't have to go back and modify our script. So we'll put the threads to sleep again while they wait for the animation to complete. A script that accomplishes this is shown below, using a simple C-like pseudocode syntax.

```
procedure DoorCinematic()
{
    thread Guy1()
    {
        // Ask guy1 to walk to the door.
        CharacterWalkToPoint(guy1, doorPosition);

        // Go to sleep until he gets there.
        WaitUntil(CHARACTER_ARRIVAL);

        // OK, we're there. Tell the other threads
        // via a signal.
        RaiseSignal("Guy1Arrived");

        // Wait for the other guy to arrive as well.
        WaitUntil(SIGNAL, "Guy2Arrived");

        // Now tell guy1 to play the "open door"
        // animation.
        CharacterAnimate(guy1, "OpenDoor");
    }
}
```

```

        WaitFor(ANIMATION_DONE);

        // OK, the door is open. Tell the other threads.
        RaiseSignal("DoorOpen");

        // Now walk thru the door.
        CharacterWalkToPoint(guy1, beyondDoorPosition);
    }

    thread Guy2()
    {
        // Ask guy2 to walk to the door.
        CharacterWalkToPoint(guy2, doorPosition);

        // Go to sleep until he gets there.
        WaitFor(CARACTER_ARRIVAL);

        // OK, we're there. Tell the other threads
        // via a signal.
        RaiseSignal("Guy2Arrived");

        // Wait for the other guy to arrive as well.
        WaitFor(SIGNAL, "Guy1Arrived");

        // Now wait until guy1 opens the door for me.
        WaitFor(SIGNAL, "DoorOpen");

        // OK, the door is open. Now walk thru the door.
        CharacterWalkToPoint(guy2, beyondDoorPosition);
    }
}

```

In the above, we assume that our hypothetical scripting language provides a simple syntax for defining threads of execution within a single function. We define two threads, one for Guy1 and one for Guy2.

The thread for Guy1 tells the character to walk to the door and then goes to sleep waiting for his arrival. We're hand-waving a bit here, but let's imagine that the scripting language magically allows a thread to go to sleep, waiting until a character in the game arrives at a target point to which he was requested to walk. In reality, this might be implemented by arranging for the character to send an event back to the script and then waking the thread up when the event arrives.

Once Guy1 arrives at the door, his thread does two things that warrant further explanation. First, it raises a signal called "Guy1Arrived." Second, it goes to sleep waiting for another signal called "Guy2Arrived." If we look at the thread for Guy2, we see a similar pattern, only reversed. This pattern of

raising a signal and then waiting for another signal is to synchronize the two threads.

In our hypothetical scripting language, a *signal* is just a Boolean flag with a name. The flag starts out false, but when a thread calls `RaiseSignal(name)`, the named flag's value changes to true. Other threads can go to sleep, waiting for a particular named signal to become true. When it does, the sleeping thread(s) wake up and continue executing. In this example, the two threads are using the “Guy1Arrived” and “Guy2Arrived” signals to synchronize with one another. Each thread raises its signal and then waits for the other thread's signal. It does not matter which signal is raised first—only when both signals have been raised will the two threads wake up. And when they do, they will be in perfect synchronization. Two possible scenarios are illustrated in Figure 16.24, one in which Guy1 arrives first, the other in which Guy2 arrives first. As you can see, the order in which the signals are raised is irrelevant, and the threads always end up in sync after both signals have been raised.

16.10 High-Level Game Flow

A game object model provides the foundations upon which a rich and entertaining collection of game object types can be implemented with which to populate our game worlds. However, by itself, a game object model only permits us to define the kinds of objects that exist in our game world and how they behave individually. It says nothing of the player's objectives, what happens if he or she completes them, and what fate should befall the player if he or she fails.

For this, we need some kind of system to control high-level game flow. This is often implemented as a finite state machine. Each state usually represents a

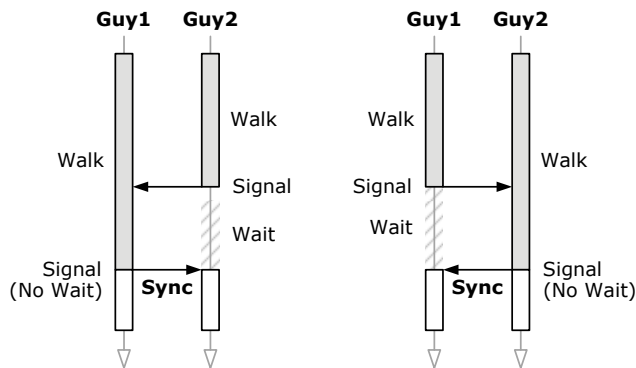


Figure 16.24. Two examples showing how a simple pattern of raising one signal and then waiting on another can be used to synchronize a pair of script threads.

single player objective or encounter and is associated with a particular locale within the virtual game world. As the player completes each task, the state machine advances to the next state, and the player is presented with a new set of goals. The state machine also defines what should happen in the event of the player's failure to accomplish the necessary tasks or objectives. Often, failure sends the player back to the beginning of the current state, so he or she can try again. Sometimes after enough failures, the player has run out of "lives" and will be sent back to the main menu, where he or she can choose to play a new game. The flow of the entire game, from the menus to the first "level" to the last, can be controlled through this high-level state machine.

The task system used in Naughty Dog's *Jak and Daxter*, *Uncharted* and *The Last of Us* franchises is an example of such a state-machine-based system. It allows for linear sequences of states (called *tasks* at Naughty Dog). It also permits parallel tasks, where one task branches out into two or more parallel tasks, which eventually merge back into the main task sequence. This parallel task feature sets the Naughty Dog task graph apart from a regular state machine, since state machines typically can only be in one state at a time.

Part V

Conclusion



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

You Mean There's More?

Congratulations! You've reached the end of your journey through the landscape of game engine architecture in one piece (and hopefully none the worse for wear). With any luck, you've learned a great deal about the major components that comprise a typical game engine. But of course, every journey's end is another's beginning. There's a great deal more to be learned about each and every topic covered within these pages. As technology and computing hardware continue to improve, more things will become possible in games—and more engine systems will be invented to support them. What's more, this book's focus was on the game engine itself. We haven't even begun to discuss the rich world of gameplay programming, a topic that could fill many more volumes.

In the following brief sections, I'll identify a few of the engine and gameplay systems we didn't have room to cover in any depth in this book, and I'll suggest some resources for those who wish to learn more about them.

17.1 Some Engine Systems We Didn't Cover

17.1.1 Movie Player

Most games include a movie player for displaying prerendered movies, also known as full-motion video (FMV). The basic components of the movie player